

TECHRISE COHORT 3

Week 3: Full-Stack Enterprise Python Project Workbook

Course 02: Backend & Data Engineering

LearnFactory Nigeria x Abia State 2026

Comprehensive Architecture blueprints & Team Deliberation Handouts

Each project is detailed in full below. The **deliberation questions** are the most important part — they are the architectural design decisions your team must debate and agree on before writing a single line of code.

Every project also includes a **Week 3 Full-Stack Requirement** section. These are not optional extras — they ensure each project exercises *all* of Week 3's core topics (OOP, decorators, generators, context managers, pytest), not just basic classes and inheritance.

PROJECT 01 — Hospital Emergency Triage System

REAL-WORLD SCENARIO

A private clinic in Aba receives walk-in patients and emergency cases every day. Right now, the nurses use a paper register. Patients wait without any priority system, so a critical case can wait behind a minor complaint. Your team must build a system that manages patients, assigns them to available doctors, tracks bed occupancy, issues prescriptions, and generates bills on discharge.

KEY CLASSES YOUR TEAM MUST DESIGN

- `Patient` — name, age, `patient_id`, `medical_history`, `current_status`
- `EmergencyPatient` → inherits from `Patient`, adds `urgency_level` (1–5)
- `Staff` → base class for all hospital workers
- `Doctor` → inherits from `Staff`, adds specialisation, `assigned_patients`
- `Nurse` → inherits from `Staff`, adds `assigned_ward`
- `Bed` — `bed_number`, `ward`, `is_occupied`, `current_patient`
- `Prescription` — `drug`, `dosage`, `frequency`, `prescribed_by`
- `Bill` — `patient`, `line_items`, `total`, `is_paid`

DELIBERATION QUESTIONS — TEAM ARCHITECTURAL DEBATE

1. Should `EmergencyPatient` inherit from `Patient`, or should `Patient` have a `priority` attribute that defaults to normal? What are the trade-offs of each approach?
2. `Doctor` and `Nurse` are both staff — but a Doctor can prescribe and a Nurse cannot. Should they both inherit from `Staff`, or should `Doctor` inherit from `Nurse` since nurses do most of what doctors do? Why is one clearly wrong?
3. When a patient is assigned to a `Bed`, who owns that relationship — does `Patient` store a reference to `Bed`, does `Bed` store a reference to `Patient`, or both? What problems does each choice create?
4. The hospital has 10 beds and all are full. An `EmergencyPatient` with `urgency_level` 5 arrives. How does your system handle this? What exception should be raised, and where?
5. Should `Prescription` belong to `Patient` (the patient receives it), to `Doctor` (the doctor issues it), or to a separate `Consultation` object that links both? Defend your answer.

OOP CONCEPTS EXERCISED

Inheritance with `super()`, polymorphism (different treatment protocols for `EmergencyPatient` vs `Patient`), encapsulation with `@property` for medical records, dunder methods (`__str__` for patient display, `__lt__` for sorting triage priority), custom exceptions.

WEEK 3 FULL-STACK REQUIREMENTS

Decorator: Write an `@admission_logger` decorator that wraps any method which admits or discharges a patient. It should log the patient name, action, and timestamp without modifying the original method. Apply it to at least two methods (e.g. `admit_patient`, `discharge_patient`). *Bonus:* write a `@requires_role("Doctor")` decorator (a decorator with arguments) that blocks a method from running unless it is called by the correct staff type, raising a `PermissionError` otherwise — use it to protect `issue_prescription()`.

Generator: Write a generator method `triage_queue()` on your `system/ward` class that yields `EmergencyPatient` objects one at a time, ordered by `urgency_level`, without building the full sorted list in memory first if the patient count is large. Explain in a comment why a generator is appropriate here even though, realistically, a clinic's queue is small — what would change if this were a citywide hospital network?

Context Manager: Write a context manager called `bed_assignment(bed, patient)` (class-based with `__enter__` / `__exit__`, or `@contextmanager`-based) that occupies a bed on entry and **guarantees** the bed is freed on exit — even if an error occurs during treatment. Demonstrate that the bed is correctly released when an exception is deliberately raised inside the `with` block.

PROJECT 02 — Smart Public Transport & Fare System

REAL-WORLD SCENARIO

A transport company in Abia State operates buses, minivans (danfos), and shared taxis on fixed routes between cities. Currently tickets are sold manually and there is no record of passengers, seat availability, or revenue per route. Your system must allow passengers to book trips, assign them to seats, calculate fares based on vehicle type and distance, and track driver assignments per trip.

KEY CLASSES YOUR TEAM MUST DESIGN

- `Vehicle` → base class with capacity, plate_number, vehicle_type
- `Bus` → inherits from `Vehicle`, larger capacity, fixed routes
- `Minivan` → inherits from `Vehicle`, flexible routes
- `SharedTaxi` → inherits from `Vehicle`, per-seat pricing
- `Driver` — name, licence_number, assigned_vehicle
- `Route` — origin, destination, distance_km, stops (list)
- `Trip` — vehicle, driver, route, departure_time, seats_available
- `Ticket` — passenger, trip, seat_number, fare, is_valid
- `Passenger` — name, phone, ticket_history
- `FareCalculator` — separate class for fare logic

🤔 DELIBERATION QUESTIONS — TEAM ARCHITECTURAL DEBATE

1. `Bus`, `Minivan`, and `SharedTaxi` all have capacity, a plate number, and can take passengers. But their fare rules are completely different. Should fare calculation live in each vehicle subclass (polymorphism), or in a separate `FareCalculator` class that takes a vehicle as input? What happens when the fare rules change?
2. A `Route` goes Aba → Umuahia → Owerri, with two stops. Does `Route` store a list of intermediate stops? How do you handle a passenger who boards at Umuahia and exits at Owerri — do they get a different ticket and fare?
3. How do you model seat assignment? Does `Trip` manage a list of seat numbers, does `Ticket` have a seat number, or does `Vehicle` own the seats? Who is responsible for checking that a seat is not double-booked?
4. A `Driver` can only drive one `Trip` at a time. How does your system prevent a `Driver` from being assigned to two simultaneous trips? Where does that validation live?
5. What should happen when a passenger tries to book a trip that is already full? Write the exception name and describe exactly what information it should carry.

🔧 OOP CONCEPTS EXERCISED

Inheritance and polymorphism across vehicle types, encapsulation of seat state in `Trip`, `@property` for available seats, `__str__` and `__repr__` for tickets and receipts, class methods for `FareCalculator`.

⚡ WEEK 3 FULL-STACK REQUIREMENTS

Decorator: Write a `@timer` decorator applied to `FareCalculator`'s fare-calculation method, printing how long the calculation took. *Bonus:* write a `@retry(times=3)` decorator (decorator with arguments) that re-attempts seat booking up to 3 times if a `SeatUnavailableError` is raised — useful for a booking system under contention.

Generator: Write a generator method `available_seats(trip)` that yields seat numbers one at a time as they are checked against the booked-seats list, instead of returning the full list at once. Also write a generator expression that computes total revenue for a `Route` by summing fares across all `Ticket` objects without building an intermediate list.

Context Manager: Write a context manager `seat_reservation(trip, seat_number)` that temporarily holds a seat during the booking process (setup) and **guarantees** the seat is released back to "available" if payment fails or any exception occurs before the booking is confirmed (cleanup). Demonstrate this with a deliberately failing payment step inside the `with` block.

PROJECT 03 — University Academic Records System

REAL-WORLD SCENARIO

A Nigerian university manages thousands of students across multiple departments. Every semester, students register for courses, sit exams, receive results, and either pass to the next level or carry over failed courses. Currently, result computation and carryover tracking are done manually in Excel and are prone to errors. Your system must automate registration, result entry, GPA computation, and carryover tracking.

KEY CLASSES YOUR TEAM MUST DESIGN

- **Student** — name, matric_no, department, level (100–500), cgpa
- **Lecturer** — name, staff_id, department, courses_taught
- **Course** — code, name, credit_units, semester, department
- **Enrollment** — student, course, semester, attendance_score
- **Result** — enrollment, exam_score, total_score, grade, grade_point
- **Department** — name, head, courses_offered
- **Semester** — academic_year, semester_number (1 or 2), is_active
- **GPA Calculator** — standalone class, computes GPA from a list of Results
- **CarryoverRecord** — student, course, original_semester, retake_count

DELIBERATION QUESTIONS — TEAM ARCHITECTURAL DEBATE

1. A student fails a course and must retake it next semester. That creates a new **Enrollment** for the same **Course**. How does your system tell the difference between a fresh enrollment and a retake? Should **Enrollment** have a **is_retake** flag, or should a **CarryoverRecord** track this separately?
2. Should **GPA Calculator** be a standalone class or a method on **Student**? What is the argument for each? (Hint: what happens when the GPA formula changes — which approach is easier to update?)
3. A course like **MTH101** (General Mathematics) is offered by the Mathematics Department but taken by students from Engineering, Computer Science, and Sciences. Does **Course** belong to one **Department**, or can it be shared? How do you model this?
4. When a student completes 100L and has no carryover courses, they are promoted to 200L. Where does this promotion logic live — in **Student**, in **Department**, or in a separate **AcademicProgress** class? Why does it matter where you put it?
5. Two universities use different grading scales (University A: 70–100 = A; University B: 75–100 = A). How do you build your **Result** class so it can support multiple grading scales without rewriting the class?

OOP CONCEPTS EXERCISED

Encapsulation of sensitive result data, `@property` for computed GPA, `__eq__` (two results are equal if same student + same course + same semester), inheritance for different student types (undergraduate, postgraduate), polymorphism for grading schemes.

WEEK 3 FULL-STACK REQUIREMENTS

Decorator: Write an `@active_semester_only` decorator that prevents `Enrollment` or `Result` records from being created or modified unless `Semester.is_active` is `True`, raising a custom exception otherwise.

Bonus: write a `@log_result_change` decorator that records every change made to a `Result` (old score, new score, who changed it, when) — useful as an audit trail for a records system where errors have real academic consequences.

Generator: Write a generator method `results_for_student(student)` on `GPACalculator` (or a related class) that yields `Result` objects one at a time across multiple semesters, rather than loading a student's entire academic history into a list before computing GPA. Explain how this generator-based approach would scale better than a list if you were computing GPA for all 5,000 students in the university at once.

Context Manager: Write a context manager `result_entry_session(lecturer, course)` that locks a course's result sheet for editing on entry (so two lecturers cannot edit it simultaneously) and **guarantees** the lock is released on exit — even if the lecturer's entry process raises an error partway through. Demonstrate the lock releasing correctly after a deliberate exception.

PROJECT 04 — Property Rental Management System

REAL-WORLD SCENARIO

Landlords in Nigerian cities rent out flats, duplexes, self-contains, shops, and warehouses. Currently, rent agreements are handwritten, payments are tracked in notebooks, and maintenance complaints are made by phone. Your system must manage property listings, tenant lease agreements, rent payment tracking (including late payments), and maintenance requests — giving landlords full visibility of their properties.

KEY CLASSES YOUR TEAM MUST DESIGN

- **Property** → base class with address, size, monthly_rent, is_available
- **Flat** → inherits from **Property**, adds num_bedrooms, floor_number
- **Duplex** → inherits from **Property**, adds num_floors, has_bq
- **SelfContain** → inherits from **Property**
- **Shop** → inherits from **Property**, adds frontage_size, business_category
- **Landlord** — name, phone, owns (list of Properties)
- **Tenant** — name, phone, occupation
- **LeaseAgreement** — tenant, property, start_date, duration_months, rent_amount
- **RentPayment** — lease, amount_paid, date_paid, status (on_time / late / defaulted)
- **MaintenanceRequest** — tenant, property, issue_description, priority, status

🤔 DELIBERATION QUESTIONS — TEAM ARCHITECTURAL DEBATE

1. A property changes tenants over time. `Flat_A` had Tenant Emeka in 2023 and Tenant Ada in 2024. If `Property` holds a reference to `Tenant`, you lose the history. If `LeaseAgreement` connects them, you keep history. As a team, argue why `LeaseAgreement` as a separate class is the better design for a real-world system.
2. A `Landlord` might use an `Agent` to manage their properties — the agent collects rent, handles complaints, and signs agreements on the landlord's behalf. Should `Agent` inherit from `Landlord` (since they do the same tasks), or should both inherit from a `Person` base class? Why does the answer matter?
3. Rent is due on the 1st of every month. If a tenant pays on the 10th, they are late. Should the system automatically compute a penalty, and where should that logic live — in `RentPayment`, in `LeaseAgreement`, or in a separate `PenaltyCalculator`?
4. What does it mean for a `Property` to be `is_available = True`? A property is available if there is no active `LeaseAgreement` referencing it. Should `is_available` be a stored attribute or a `@property` that computes it dynamically from lease data? Which is safer and why?
5. A `MaintenanceRequest` for a burst pipe is urgent; a request to repaint a wall is low priority. Both are maintenance. How does your team model urgency — as a string ("low/medium/high"), as an integer (1–5), or as a separate `Priority` class? What are the trade-offs?

🔧 OOP CONCEPTS EXERCISED

Multi-level inheritance (`Property` → `Flat`, `Duplex`, etc.), `@property` for `is_available` and `total_rent_owed`, encapsulation of payment status, `__str__` for lease printing, dunder `__eq__` for comparing leases, custom exceptions for double-booking.

⚡ WEEK 3 FULL-STACK REQUIREMENTS

Decorator: Write a `@validate_positive` decorator applied to any method that accepts a monetary amount (e.g. `record_payment(amount)`), raising `ValueError` for zero or negative amounts. *Bonus:* write a `@landlord_only` decorator (decorator with arguments, e.g. `@requires_role("Landlord")`) that restricts methods like `terminate_lease()` to landlord/agent callers.

Generator: Write a generator method `overdue_payments(landlord)` that yields `RentPayment` records one at a time across all of a landlord's properties where `status` is "late" or "defaulted", instead of building the full list upfront. Use it to print a rent-collection report without loading every payment record into memory at once.

Context Manager: Write a context manager `lease_signing(tenant, property)` that marks a property as "pending" on entry (so it cannot be double-booked while paperwork is being finalised) and **guarantees** it reverts to "available" on exit if the lease signing fails for any reason. Demonstrate correct rollback when an exception interrupts the signing process.

PROJECT 05 — Supermarket Inventory and Point-of-Sale System

REAL-WORLD SCENARIO

A medium-sized supermarket in Aba has over 500 products across different categories. The owner loses money every month due to stockouts (items run out without warning), overpriced goods (price not updated after supplier changes), and cashier errors. Your system must handle product inventory, process sales at checkout, calculate discounts, generate receipts, alert when stock is low, and track supplier purchase orders.

KEY CLASSES YOUR TEAM MUST DESIGN

- **Product** — name, sku, price, category, quantity_in_stock, reorder_level
- **PerishableProduct** → inherits from **Product**, adds expiry_date
- **Category** — name, tax_rate
- **Inventory** — manages all products, handles restocking alerts
- **Sale** — cashier, timestamp, items (list of SaleItems), status
- **SaleItem** — product, quantity, unit_price, discount_applied
- **Receipt** — sale, total_before_tax, tax, total_after_tax
- **Staff** → base class
- **Cashier** → inherits from **Staff**
- **Manager** → inherits from **Staff**, adds `void_sale()`, `apply_discount()` permissions
- **Supplier** — name, contact, products_supplied
- **PurchaseOrder** — supplier, items, order_date, delivery_status

🤔 DELIBERATION QUESTIONS — TEAM ARCHITECTURAL DEBATE

1. Some products expire (food, medicine) and some do not (electronics, clothing). Should `PerishableProduct` inherit from `Product` adding an `expiry_date`, or should `Product` have an optional `expiry_date=None`? What problem does each design create when you need to check for expired stock across all products?
2. When a cashier scans a product and it is out of stock, who raises the error — the `Sale` object, the `SaleItem` or `Inventory`? Write the exact exception name and message your team agrees on, and explain where in the call chain it gets raised.
3. Discounts come in three types: a flat amount off (₦500 off), a percentage off (10%), and buy-one-get-one. Should these be subclasses of a `Discount` base class, or should `SaleItem` have a `discount_type` string and `discount_value` number? What happens when a fourth discount type is needed in the future?
4. A `Manager` can void a sale and apply discounts, but a `Cashier` cannot. Should `Manager` inherit from `Cashier` (since they can also do what cashiers do), or should both inherit from `Staff`? What real-world problems arise from `Manager` inheriting from `Cashier`?
5. `Inventory` should automatically alert when a product's stock falls below its `reorder_level`. Should `Inventory.restock_alert()` be called manually after every sale, or should the `__setattr__` dunder trigger it automatically when `quantity_in_stock` changes? Which is safer in a real system?

🔧 OOP CONCEPTS EXERCISED

Inheritance and polymorphism (`Cashier` vs `Manager` permissions, `Product` vs `PerishableProduct`), encapsulation of stock levels, `@property` for total price with discount, `__str__` for receipt output, `__len__` for number of items in a sale, custom exceptions for out-of-stock and expired products.

⚡ WEEK 3 FULL-STACK REQUIREMENTS

Decorator: Write a `@manager_only` decorator (decorator with arguments, e.g. `@requires_role("Manager")`) that protects `void_sale()` and `apply_discount()`, raising `PermissionError` if a `Cashier` calls them. *Bonus:* write a `@timer` decorator around the checkout method to measure how long a sale takes to process — useful for spotting slow checkout lanes.

Generator: Write a generator method `low_stock_products(inventory)` that yields `Product` objects one at a time where `quantity_in_stock < reorder_level`, scanning across 500+ products without building the full filtered list first. Also write a generator expression that computes total inventory value (sum of `price * quantity_in_stock` across all products) lazily.

Context Manager: Write a context manager `sale_transaction(cashier)` that opens a new `Sale` on entry and **guarantees** the sale is marked "voided" (not left in a half-completed state) if any error occurs while scanning items or applying discounts — only marking it "completed" if the `with` block finishes without error. Demonstrate this with a deliberate stock error mid-sale.

PROJECT 06 — Football League Management System

REAL-WORLD SCENARIO

A local football association in Abia State runs a 12-team league every season. Right now, results are posted on WhatsApp, the league table is computed manually every week, and player transfers between clubs are tracked in a spreadsheet that everyone argues about. Your system must manage clubs, register players with their positions, record match results, automatically update the league table, track goals and cards per player, and process player transfers between clubs.

KEY CLASSES YOUR TEAM MUST DESIGN

- **Club** — name, city, stadium, founded_year, budget
- **Player** → base class with name, age, club, nationality, market_value
- **Goalkeeper** → inherits from **Player**, adds clean_sheets
- **OutfieldPlayer** → inherits from **Player**, adds preferred_foot
- **Forward** → inherits from **OutfieldPlayer**, adds goals_scored
- **Midfielder** → inherits from **OutfieldPlayer**, adds assists
- **Defender** → inherits from **OutfieldPlayer**, adds tackles_won
- **Match** — home_club, away_club, date, venue, status (scheduled/live/completed/postponed)
- **MatchResult** — match, home_goals, away_goals, scorers, cards_issued
- **Goal** — scorer (Player), match, minute, is_own_goal
- **Card** — player, match, minute, card_type (yellow/red)
- **Transfer** — player, from_club, to_club, fee, transfer_date, transfer_type (permanent/loan)
- **Season** — year, matches (list), clubs (list)
- **LeagueTable** — season, standings (computed from results)

🤔 DELIBERATION QUESTIONS — TEAM ARCHITECTURAL DEBATE

1. Should `Forward`, `Midfielder`, and `Defender` all inherit directly from `Player`, or should there be a middle class `OutfieldPlayer` that they all inherit from — with `Goalkeeper` also inheriting directly from `Player`? Draw the tree on paper, argue both designs, and agree on one. What does `OutfieldPlayer` give you that makes the code cleaner?
2. A player scores an own goal. The goal goes in the opponent's tally but it is still attached to the player who scored it. How does `Goal` model this? Should `is_own_goal` change which club's total is incremented, and where does that logic live — in `Goal`, in `MatchResult`, or in `LeagueTable`?
3. The league table is updated after every match. `LeagueTable` needs wins, draws, losses, goals for, goals against, and points for each club. Should these be stored as data in `LeagueTable` (updated after each result), or should `LeagueTable` compute them fresh every time by reading through all `MatchResult` objects? Which approach is more accurate? Which is faster?
4. A player is transferred on loan — they play for `Club B` for 6 months but still belong to `Club A`. During the loan, which club shows up when you call `player.club`? How does your `Transfer` class distinguish a permanent move from a loan, and how does a loan return (going back to `Club A`) get recorded?
5. A match can be scheduled, live, completed, or postponed. A `LeagueTable` should only count completed matches. Where do you put the validation — does `LeagueTable` check `match.status == "completed"` before reading results, or does `MatchResult` only exist for completed matches? As a team, define the rule and enforce it in code.

🔧 OOP CONCEPTS EXERCISED

Multi-level inheritance (`Player` → `OutfieldPlayer` → `Forward/Midfielder/Defender`), polymorphism (`speak()` equivalent is `player_info()` returning position-specific stats), dunder methods (`__str__` for scorecards, `__lt__` for league table sorting, `__eq__` for transfer equality), `@property` for total points in `LeagueTable`, encapsulation of player market value.

⚡ WEEK 3 FULL-STACK REQUIREMENTS

Decorator: Write an `@completed_matches_only` decorator that prevents any method on `MatchResult` or `LeagueTable` from processing a `Match` unless `match.status == "completed"`, raising a custom exception otherwise. *Bonus:* write a `@debug` decorator applied to `Transfer`'s creation method, logging the player, clubs, and fee whenever a transfer is recorded.

Generator: Write a generator method `season_goals(season)` that yields `Goal` objects one at a time across every match in a season, instead of collecting them into a list first — useful for computing a top-scorer leaderboard without holding every goal in memory at once. Also write a generator expression that sums total goals for a specific `Player` across the season.

Context Manager: Write a context manager `match_recording(match)` that sets `match.status = "live"` on entry and **guarantees** the match is correctly set to `"completed"` (on success) or rolled back to `"scheduled"` (if an error interrupts recording, e.g. a connectivity failure mid-match) on exit — never leaving a match stuck in "live" state. Demonstrate this with a deliberate error during result entry.



TUTOR DELIVERY NOTES

Each project is best presented in two stages:

- **Stage 1 — Deliberation session:** Give the team the scenario and deliberation questions only. Do not show them the class list or the Week 3 Full-Stack Requirement. Let them argue and propose their own structure first. This is where the real thinking happens.
- **Stage 2 — Guided design:** After 20–30 minutes of debate, introduce the suggested class list and compare it to what the team proposed. Discuss why certain designs are better than others. Only once the class design is settled should you introduce the **Week 3 Full-Stack Requirement** — present it as "now make it production-grade," reinforcing that real systems need logging/validation (decorators), memory-efficient iteration (generators), and guaranteed cleanup (context managers) on top of good class design.

The deliberation questions are designed so there is no single correct answer — each has genuine trade-offs. Teams that argue well and commit to a design (even an imperfect one) will learn more than teams that are given the answer. The goal is not a perfect system. The goal is to think like engineers.

Every project's Full-Stack Requirement is intentionally scoped to be demonstrable in a short demo: one working decorator (plus a bonus decorator-with-arguments), one generator that visibly avoids building a full list, and one context manager that visibly survives a deliberate error. Teams should be ready to show all three live, and to explain *why* each tool was the right one for that specific job — not just that it was used.